

Critical Reproduction and Stress-Test of a Machine-Learning Credit-Card Fraud Detector

Data Science in Cyber — Final Project

Dr. Uri Itai

Source under evaluation:

shakiliitju / Credit-Card-Fraud-Detection-Using-Machine-Learning

Author	Itay Cohen — 328316591
Submission	Data Science in Cyber, 2026
Repository	<a href="https://github.com/itaychn137/<your-new-repo>">https://github.com/itaychn137/<your-new-repo>
Dataset	Kaggle / ULB credit-card fraud (Dal Pozzolo et al.)

1. Summary of the Source

Problem. The source addresses credit-card fraud detection: given a single card transaction, decide whether it is fraudulent or legitimate. This is a binary classification problem on highly imbalanced data, because genuine fraud is rare.

Why it matters. Card fraud causes large direct financial losses and erodes customer trust, and the volume of transactions makes manual review impossible. An automated detector that flags suspicious transactions in real time is therefore valuable — but because fraud is rare, the cost of the two error types is very asymmetric, which makes the choice of success measure central.

Proposed solution. The source trains standard supervised classifiers — a k-nearest-neighbour model and a decision tree — on the public dataset, scales the features, performs a train/test split, and reports test accuracy. Its headline result is approximately 100% accuracy, presented as evidence that the models detect fraud effectively.

Dataset. The Kaggle / ULB credit-card dataset contains 284,807 European card transactions from September 2013, of which only 492 (0.172%) are fraudulent. For confidentiality the original features are not released: 28 of the columns (V1–V28) are anonymised principal components produced by PCA, and only Time (seconds since the first transaction) and Amount are given in the clear, plus the Class label.

Methodology employed. Features are standardised, the data is split into train and test sets, the KNN ($k = 1$) and decision-tree models are fitted, and performance is summarised with accuracy. No imbalance handling (resampling or class weighting) is applied, and no metric beyond accuracy is reported.

2. Critical Evaluation

Main claim. The author claims that achieving ~100% accuracy demonstrates an effective fraud detector.

Is the claim supported by the evidence? No. On a dataset that is 99.83% legitimate, accuracy is dominated by the majority class. A model that predicts “legitimate” for every transaction already scores 99.83% accuracy without detecting a single fraud, so the reported ~100% is barely above doing nothing. We reproduced the source’s pipeline and obtained 99.95% accuracy — yet the same model misses real frauds, and when judged by a metric appropriate to fraud (PR-AUC) the source’s two models are the weakest we tested (Decision Tree 0.56, KNN 0.71).

Is the evaluation methodology appropriate? No. Accuracy is the wrong yardstick for a rare-event problem. Appropriate measures are precision, recall, F1/F2, Matthews Correlation Coefficient, and especially the precision–recall AUC, which react to the minority class. ROC-AUC is also of limited use here because it is insensitive to prevalence. The source reports none of these.

Weaknesses and limitations. (i) accuracy-only evaluation on an extremely imbalanced problem; (ii) no imbalance handling — no class weights, resampling, or threshold tuning; (iii) $k = 1$ nearest neighbour, which simply memorises the training data and generalises poorly; (iv) the scaler is fit on the whole dataset before the train/test split, leaking test statistics into preprocessing; (v) exact duplicate rows that can straddle the train/test boundary. We quantify the impact of (iv) and (v) in Section 5.

Are the conclusions justified? No. The conclusion that the models detect fraud well is not supported. The headline number is an artefact of the metric, not a property of the detector. This

does not mean the task is hopeless — our corrected models reach a genuine PR-AUC of about 0.87 — but that is a different, more honest claim than the one the source makes.

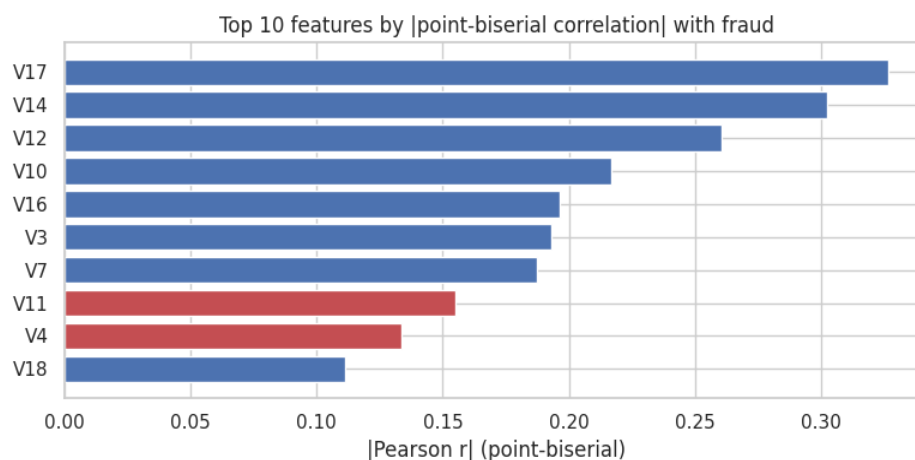
3. Feature Engineering Analysis

Was feature engineering performed? Largely upstream and outside the author's control: 28 of the 30 inputs are already PCA components, centred and scaled by construction. Only Amount and Time are on raw scales. The source's own engineering is limited to standardising the columns.

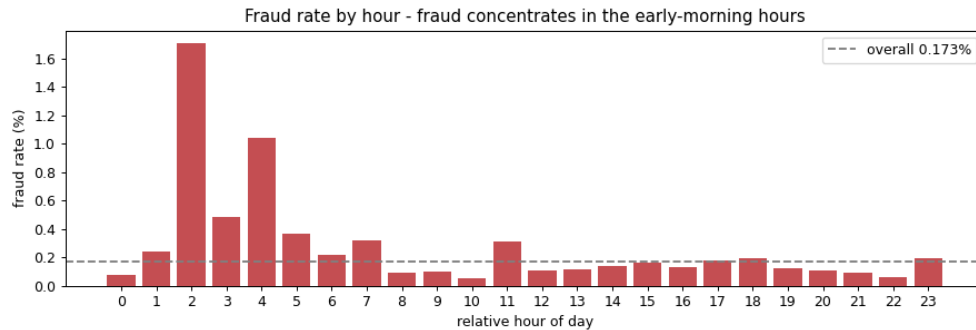
Encoding and scaling. No categorical encoding is needed because every feature is numeric. Scaling matters for distance- and gradient-based models (KNN, logistic regression) but not for tree ensembles. The important point is procedural: the scaler must be fit on the training data only. The source fits it on the full dataset, which is a leak (assessed in Section 5).

Redundancy. The dataset contains 1,081 exact duplicate rows. Redundancy of this kind is spotted with a duplicate check on the rows (and, for columns, by inspecting pairwise identity or near-constant variance); it is tackled by de-duplicating before splitting so that identical rows cannot appear in both train and test.

Feature ranking and correlation method. To identify the most fraud-associated features we used the **point-biserial correlation** (Pearson with a binary target), which is the correct measure for a continuous feature against a binary outcome. Spearman, although robust to outliers, is inappropriate here: with 99.83% of rows tied at Class = 0, the target's ranks are dominated by one block of ties and every correlation collapses toward zero. Under the correct measure, V17, V14, V12 and V10 emerge as the strongest signals.



Temporal analysis. Because Time is a temporal feature, we examined the fraud rate over a relative hour-of-day cycle (Hour = Time // 3600 mod 24). The rate swings from about 0.05% at its lowest to about 1.7% in the small hours — a roughly 35-fold difference that matches world knowledge: fraud slips through more easily when cardholders are asleep. The source ignores Time entirely.



Feature creation (measured, not assumed). We engineered two interpretable features — Hour (above) and `log_Amount` (a log transform for Amount's heavy right tail) — and measured their effect. Adding both moved XGBoost's PR-AUC only from 0.872 to 0.869. `log_Amount` receives essentially zero importance because tree models are invariant to monotonic transforms (a log helps linear or distance-based models, not gradient-boosted trees); Hour carries a clear EDA pattern but barely lifts the model because the PCA components already encode most of the separable structure. The honest conclusion is that feature creation aids understanding more than performance on this data.

Was the process meaningful, and what could be added? Because the features are anonymised PCA components, their cybersecurity interpretation is limited — we cannot say “hyphens in a domain” as in a URL problem. In a non-anonymised setting, strong additional features would be behavioural aggregates: transaction velocity per card, deviation from a card's typical amount, time-of-day patterns, and merchant-category risk. These are unavailable here precisely because of the privacy-preserving PCA transform.

4. Reproducibility Analysis

Can the code be executed? Yes. We reproduced the source's approach on the real data and obtained 99.95% accuracy, consistent with the claimed ~100% (the difference is rounding).

Are files and dependencies available? The dataset is public on Kaggle. Our reproduction repository commits a gzipped copy of the CSV (kept under the platform file-size limit), a requirements file, and a notebook that clones the repository so the relative data path resolves in a fresh environment.

Hidden preprocessing steps. Two are easy to miss: the scaler is fit before the split (a leak), and the source does not fix a random seed for its split, so its exact numbers are not deterministic. Our reproduction fixes a seed and keeps preprocessing inside a pipeline.

Overall reproducibility. The work is reproducible in the narrow sense that the code runs and the accuracy can be re-obtained. It is not trustworthy in the scientific sense, because the reproduced number does not measure what the author claims it measures.

5. Experimental Results

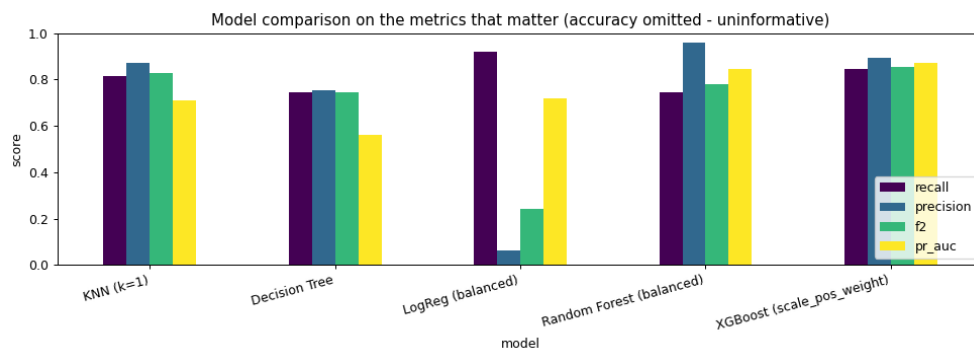
We trained four models on one stratified 80/20 split and evaluated them with the full metric suite. Scaling was done inside a pipeline (fit on training data only); imbalance was handled with class weights or `scale_pos_weight` rather than resampling before the split.

model	accuracy	precision	recall	F1	F2	PR-AUC
-------	----------	-----------	--------	----	----	--------

XGBoost (scale_pos_weight)	0.9996	0.893	0.847	0.869	0.856	0.872
Random Forest (balanced)	0.9995	0.960	0.745	0.839	0.780	0.845
LogReg (balanced)	0.9755	0.061	0.918	0.114	0.241	0.719
KNN k=1 (source's model)	0.9995	0.870	0.816	0.842	0.826	0.710
Decision Tree (source's model)	0.9991	0.753	0.745	0.749	0.746	0.561

Table 1. Model comparison on the held-out test set (positive class = fraud).

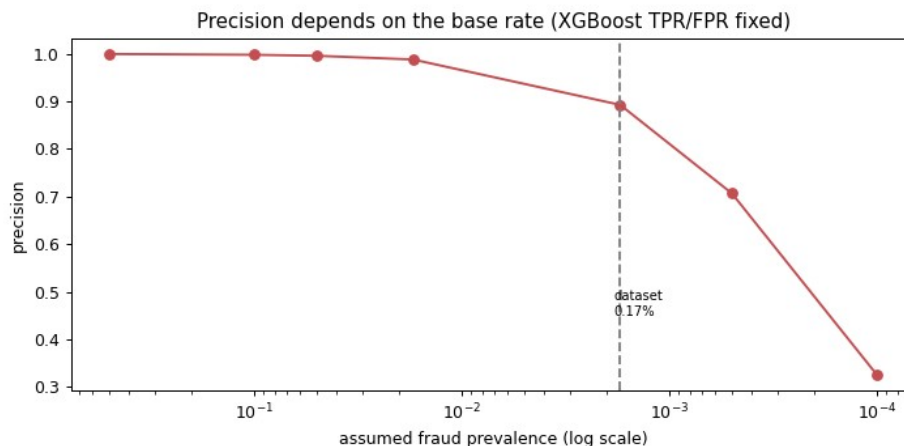
Reading the table. Accuracy is ~0.999 for every model except balanced logistic regression — whose accuracy actually drops to 0.975 precisely because it catches the most fraud (recall 0.92) at the cost of flooding analysts with false alarms (precision 0.06). Accuracy therefore cannot rank these models; PR-AUC can, and it puts XGBoost first (0.872) and both of the source's models last (KNN 0.710, Decision Tree 0.561).



Cross-validation. A single split leaves only ~98 frauds in the test set, so we verified stability with 5-fold stratified cross-validation of XGBoost: PR-AUC 0.859 ± 0.024 (folds 0.82–0.89), with the single-split 0.872 inside that range. The headline is stable, not the product of one lucky split.

Experiment 1 — The accuracy illusion

A model that never predicts fraud scores 99.83% accuracy, so the source's headline is ~0.1 points above doing nothing. The precision a model achieves also depends on the base rate it is deployed at: holding XGBoost's true- and false-positive rates fixed, precision stays high at the dataset's 0.17% prevalence (~0.89) but would fall to about 0.33 if fraud were ten times rarer.

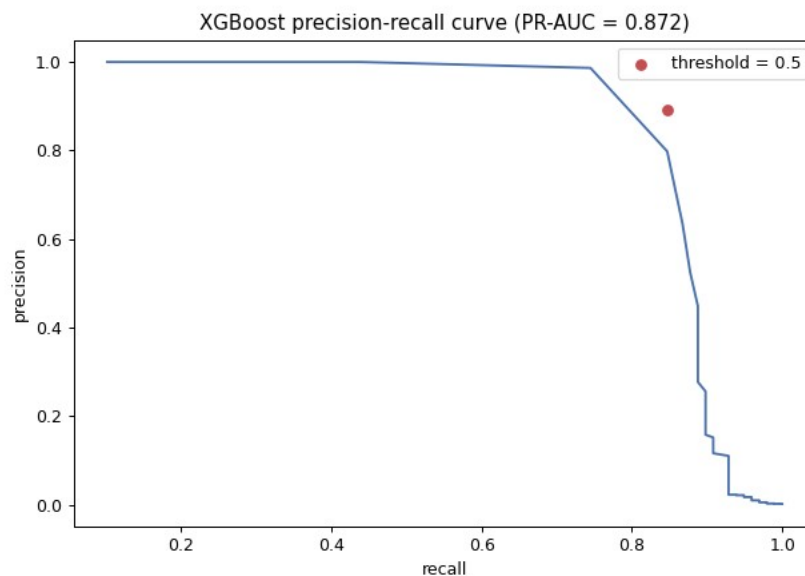


Experiment 2 — Leakage and duplicate rows

We measured, rather than assumed, the impact of the two methodological flaws. Fitting the scaler on the full dataset before splitting produced results identical to the correct pipeline: on 285,000 rows the scaler statistics are indistinguishable whether or not the test set is included, so the leak is real malpractice but has negligible numeric effect here. The 1,081 duplicate rows cause only mild contamination (0.6% of test rows have a twin in train, and there are no contradictory feature vectors); de-duplicating moves XGBoost's PR-AUC from 0.872 to 0.830. Reporting these as small effects — rather than inflating them — is itself part of an honest evaluation.

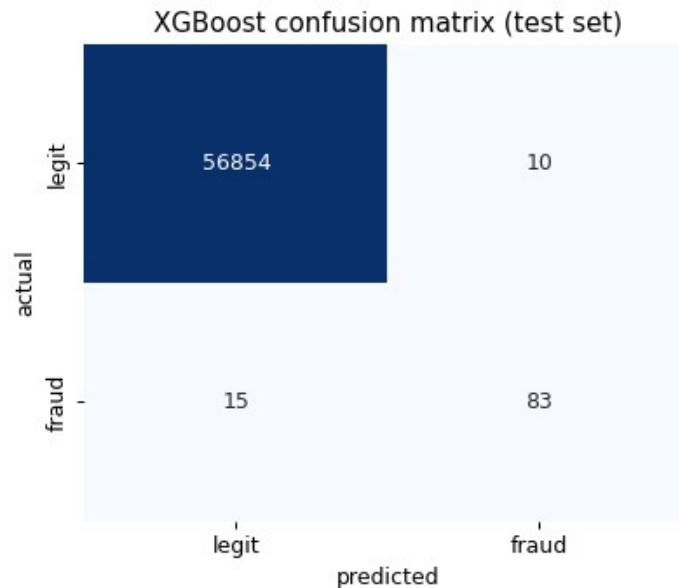
Experiment 3 — $k = 1$ memorisation and imbalance handling

The source's $k = 1$ nearest-neighbour model memorises the training set. A trivial change to $k = 5$ already lifts PR-AUC from 0.710 to 0.841, and an imbalance-aware XGBoost reaches 0.872 while exposing a tunable precision/recall trade-off. The precision–recall curve below shows that trade-off and the default-threshold operating point; a single accuracy number encodes none of it.



Error analysis

On the test set XGBoost catches 83 of 98 frauds, misses 15, and raises 10 false alarms. The missed frauds skew toward small amounts and receive very low fraud scores (maximum 0.146, well below the 0.5 threshold), meaning the model finds them genuinely indistinguishable from ordinary small purchases — they are hard errors, not threshold artefacts. Catching them would require lowering the threshold so far (recall 0.90 → precision ~0.16) that five of every six alerts would be false. The right operating point is a business decision about the cost of a missed fraud versus a reviewed false alarm — exactly the judgement a lone accuracy figure cannot support.



6. Conclusions

Key findings. (1) Accuracy is meaningless on this 0.17%-fraud problem; the source's ~100% is barely above a do-nothing baseline, and both its models (KNN, Decision Tree) are the weakest by PR-AUC. (2) The source's leakage and duplicate issues are real but, on this large dataset, numerically minor. (3) The model and metric choices are poor: a proper, imbalance-aware model roughly doubles the achievable PR-AUC the source never measured.

Lessons learned. In cybersecurity machine learning, where the events that matter are rare, the choice of evaluation metric is itself a result. Rigour also means measuring the impact of a suspected flaw rather than assuming it — here a real leak turned out to be harmless, which is a finding worth reporting.

Strengths and weaknesses of the source. Its strengths are a clean, runnable pipeline on a well-known dataset. Its weaknesses are an inappropriate success metric, no imbalance handling, a memorising model, and preprocessing that leaks (if harmlessly). The conclusion of success is not justified.

Suggested improvements. report precision/recall/F2/MCC/PR-AUC instead of accuracy; handle imbalance via class weights or correctly-placed resampling; choose the model and threshold by the precision/recall trade-off and the business cost of errors; keep all preprocessing inside a cross-validated pipeline; and de-duplicate before splitting.

7. Executive Summary

This project critically evaluates a published machine-learning tutorial that detects credit-card fraud and claims approximately 100% accuracy. The dataset is the Kaggle / ULB collection of 284,807 transactions with only 492 frauds (0.172%), described by 28 anonymised PCA features plus Time and Amount.

We reproduced the source on the real data and stress-tested its central claim. We confirmed that the ~100% accuracy is reproducible (99.95%) but meaningless: a model that never flags fraud already scores 99.83%, and the source's two models are the weakest by precision–recall

AUC (Decision Tree 0.56, KNN 0.71). We then ran three experiments — the accuracy illusion and base-rate sensitivity, data leakage and duplicate contamination, and the effect of the $k = 1$ choice and imbalance handling — and trained honest baselines (class-weighted logistic regression and random forest, and an imbalance-aware XGBoost).

The author's leakage and duplicate issues proved real but numerically minor on this large dataset, which we report honestly rather than overstate. The substantive problems are the metric and the model: judged correctly, the genuine performance is a PR-AUC of about 0.87, achieved by XGBoost with a tunable precision/recall trade-off. The author's claim of success is therefore not supported by appropriate evidence, even though the underlying task is learnable. The key insight is methodological: under extreme class imbalance, the choice of evaluation metric determines whether a result is real.

8. Summing It Up

Problem and source. We evaluated shakiliitju's credit-card fraud detector, which trains KNN and decision-tree models on the ULB dataset and reports ~100% accuracy.

Dataset and methodology. 284,807 transactions, 0.172% fraud, anonymised PCA features. We reproduced the pipeline, applied a full metric suite, ran three targeted experiments, and compared four models under correct preprocessing and imbalance handling.

Were the author's claims supported? No. The accuracy is reproducible but does not measure fraud detection; on a 0.17%-positive problem accuracy is dominated by the legitimate class, and the source's model is the weakest by the metrics that matter.

Most important insights. Pick the metric for the problem (PR-AUC and the precision/recall trade-off, not accuracy); measure the impact of suspected flaws rather than assuming it; and recognise that simple choices — k , class weights, threshold — matter more than the headline suggests.

Do we recommend this source as a template? Not as published — its evaluation would give a false sense of security on any imbalanced security task. With the corrections in this report, the dataset and overall approach are a reasonable starting point.

Final conclusion. The fraud-detection task is genuinely learnable (PR-AUC ~0.87), but the source's claim of success rests on the wrong yardstick. In rare-event cybersecurity problems, the evaluation metric is the result.